

**SIMATS ENGINEERING
ASSESSMENT TOOL 2**

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO5: Analyze object-oriented software using quality assurance and usability testing
Techniques (BL4, BL5)
Assessment Tool Weightage: 25%

QUESTIONS: 05

Total Marks:25

Annexure A

Error Analysis Task

Code of Conduct:

*I Sayed Fazal(192311291) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Sd.Fazal..

Error Analysis Task

Q1. Scenario: Online Banking System

Modules:

User Login, Fund Transfer, Account Verification

Question:

Write pseudocode for a quality assurance testing process that validates secure user authentication and fund transfer operations. Include input validation, transaction verification, exception handling, and test result logging.

ANSWER :

BEGIN QA_Testing

Initialize TestLog

FOR each TestCase in Authentication_TestCases DO

 Start Timer

 INPUT username

 INPUT password

 IF username is empty OR password is empty THEN

 LogResult(TestCase, "FAIL", "Invalid or missing input")

 CONTINUE

 END IF

 TRY

 result ← AuthenticateUser(username, password)

 IF result = SUCCESS THEN

 LogResult(TestCase, "PASS", "Authentication successful")

 ELSE

 LogResult(TestCase, "PASS", "Invalid login correctly rejected")

 END IF

CATCH AuthenticationException

LogResult(TestCase, "PASS", "Authentication exception handled")

CATCH Exception

LogResult(TestCase, "FAIL", "Unexpected exception")

END TRY

END FOR

FOR each TransferTest in FundTransfer_TestCases DO

INPUT senderAccount

INPUT receiverAccount

INPUT amoun

IF senderAccount is empty OR receiverAccount is empty THEN

LogResult(TransferTest, "FAIL", "Missing account information")

CONTINUE

END IF

IF amount <= 0 THEN

LogResult(TransferTest, "PASS", "Invalid amount rejected")

CONTINUE

END IF

TRY

VerifyAccount(senderAccount)

VerifyAccount(receiverAccount)

IF senderAccount = receiverAccount THEN

LogResult(TransferTest, "PASS", "Same-account transfer rejected")

CONTINUE

END IF

IF GetBalance(senderAccount) < amount THEN

LogResult(TransferTest, "PASS", "Insufficient balance rejected")

CONTINUE

END IF

transaction $\leftarrow$ TransferFunds(senderAccount, receiverAccount, amount)

IF VerifyTransaction(transaction) = TRUE THEN

LogResult(TransferTest, "PASS", "Fund transfer verified")

ELSE

LogResult(TransferTest, "FAIL", "Transaction verification failed")

END IF

CATCH AccountException

LogResult(TransferTest, "PASS", "Account exception handled")

CATCH TransactionException

LogResult(TransferTest, "PASS", "Transaction exception handled")

CATCH Exception

LogResult(TransferTest, "FAIL", "Unexpected exception")

END TRY

END FOR

SaveTestLog(TestLog)

GenerateTestReport(TestLog)

END QA_Testing

Q2. Scenario: E-Commerce Shopping Platform

Features:

Product Search, Shopping Cart, Payment Gateway

Question:

Design pseudocode for a usability testing evaluation that measures customer interaction efficiency, including product search time, checkout completion time, and navigation complexity. Explain how the algorithm collects and analyzes user behavior data.

ANSWER :

BEGIN Usability_Test

Initialize SearchTimes

Initialize CheckoutTimes

Initialize NavigationScores

FOR each User in TestUsers DO

StartTimer()

User searches for a product

SearchTime $\leftarrow$ StopTimer()

Store(SearchTimes, SearchTime)

StartTimer()

User selects product

User adds product to cart

User proceeds to checkout

User completes payment

CheckoutTime $\leftarrow$ StopTimer()

Store(CheckoutTimes, CheckoutTime)

NavigationSteps $\leftarrow$ CountUserNavigationSteps(User)

NavigationComplexity $\leftarrow$ CalculateNavigationComplexity(
 NavigationSteps,
 NumberOfPagesVisited
)

Store(NavigationScores, NavigationComplexity)

RecordUserActions(

```
        searchQueries,  
        clicks,  
        pagesVisited,  
        cartActions,  
        checkoutActions  
    )
```

```
END FOR
```

```
AverageSearchTime ← Average(SearchTimes)
```

```
AverageCheckoutTime ← Average(CheckoutTimes)
```

```
AverageNavigationComplexity ← Average(NavigationScores)
```

```
Display("Average Product Search Time:",  
        AverageSearchTime)
```

```
Display("Average Checkout Time:",  
        AverageCheckoutTime)
```

```
Display("Average Navigation Complexity:",  
        AverageNavigationComplexity)
```

```
IF AverageSearchTime > SearchThreshold THEN  
    Report("Product search needs improvement")
```

END IF

IF AverageCheckoutTime > CheckoutThreshold THEN

 Report("Checkout process needs improvement")

END IF

IF AverageNavigationComplexity > NavigationThreshold THEN

 Report("Navigation should be simplified")

END IF

GenerateUsabilityReport()

END Usability_Test

Q3. Scenario: Hospital Management System

Modules:

Patient Registration, Appointment Scheduling, Medical Records

Question:

Write pseudocode to generate and execute test cases for appointment scheduling functionality. Include validation of doctor availability, appointment conflicts, patient information accuracy, and system responses to invalid requests.

ANSWER :

BEGIN Appointment_Testing

 TestCases ← GenerateTestCases()

 FOR each TestCase in TestCases DO

Display("Executing:", TestCase.ID)

patient ← TestCase.patient

doctor ← TestCase.doctor

date ← TestCase.date

time ← TestCase.time

TRY

IF patient is NULL THEN

Expected ← "Invalid patient"

Actual ← ScheduleAppointment(patient, doctor, date, time)

Verify(Expected, Actual)

CONTINUE

END IF

IF PatientInformationIsValid(patient) THEN

Expected ← "Patient information error"

Actual ← ScheduleAppointment(patient, doctor, date, time)

Verify(Expected, Actual)

CONTINUE

END IF

IF NOT DoctorExists(doctor) THEN

Expected ← "Doctor not found"

Actual ← ScheduleAppointment(patient, doctor, date, time)

Verify(Expected, Actual)

CONTINUE

END IF

IF NOT DoctorAvailable(doctor, date, time) THEN

Expected ← "Doctor unavailable"

Actual ← ScheduleAppointment(patient, doctor, date, time)

Verify(Expected, Actual)

CONTINUE

END IF

IF AppointmentConflict(patient, date, time) THEN

Expected ← "Appointment conflict"

Actual ← ScheduleAppointment(patient, doctor, date, time)

Verify(Expected, Actual)

CONTINUE

END IF

Actual ← ScheduleAppointment(patient, doctor, date, time)

IF Actual = SUCCESS THEN

Log(TestCase.ID, "PASS",
"Appointment successfully scheduled")

ELSE

Log(TestCase.ID, "FAIL",
"Unexpected scheduling response")

END IF

CATCH Exception

Log(TestCase.ID, "PASS",
"Invalid request handled without system failure")

END TRY

END FOR

GenerateTestReport()

END Appointment_Testing

Q4. Scenario: Airline Reservation System

Modules:

Passenger Registration, Flight Booking, Seat Allocation

Question:

Develop pseudocode for a constraint-based testing approach where booking requests must satisfy conditions such as valid passenger information, seat availability, and payment confirmation. Show how invalid booking scenarios are detected and handled.

ANSWER :

BEGIN Constraint_Based_Testing

TestCases ← GenerateBookingTestCases()

FOR each Booking in TestCases DO

 validPassenger ← ValidatePassenger(
 Booking.passenger
)

 validFlight ← ValidateFlight(
 Booking.flight
)

 availableSeat ← CheckSeatAvailability(
 Booking.flight,
 Booking.seat
)

 paymentConfirmed ← VerifyPayment(
 Booking.payment
)

IF NOT validPassenger THEN

 RejectBooking(Booking)

 Log(Booking.ID, "PASS",

 "Invalid passenger information detected")

 CONTINUE

END IF

IF NOT validFlight THEN

 RejectBooking(Booking)

 Log(Booking.ID, "PASS",

 "Invalid flight detected")

 CONTINUE

END IF

IF NOT availableSeat THEN

 RejectBooking(Booking)

 Log(Booking.ID, "PASS",

 "Unavailable seat detected")

 CONTINUE

END IF

IF NOT paymentConfirmed THEN

 RejectBooking(Booking)

 Log(Booking.ID, "PASS",

"Payment failure detected")

CONTINUE

END IF

TRY

reservation ← CreateBooking(
 Booking.passenger,
 Booking.flight,
 Booking.seat
)

IF reservation = SUCCESS THEN

Log(Booking.ID, "PASS",
 "Booking successfully created")

ELSE

Log(Booking.ID, "FAIL",
 "Booking creation failed")

END IF

CATCH BookingException

Log(Booking.ID, "PASS",
 "Booking exception handled")

END TRY

END FOR

GenerateConstraintTestReport()

END Constraint_Based_Testing

Q5. Scenario: Smart Library Management System

Features:

Book Search, Book Reservation, Borrow/Return Services

Question:

Write pseudocode for integrating quality assurance and usability testing where the system evaluates both functional correctness (reservation and borrowing operations) and user experience factors such as response time, ease of navigation, and accessibility.

BEGIN Library_QA_Usability_Test

Initialize TestResults

Initialize ResponseTimes

Initialize NavigationScores

Initialize AccessibilityResults

FOR each TestCase in LibraryTestCases DO

StartTimer()

IF TestCase.Type = "SEARCH" THEN

result ← SearchBook(TestCase.Keyword)

IF result is correct THEN

 FunctionalResult ← "PASS"

ELSE

 FunctionalResult ← "FAIL"

END IF

ELSE IF TestCase.Type = "RESERVATION" THEN

 result ← ReserveBook(
 TestCase.User,
 TestCase.Book
)

IF result = SUCCESS THEN

 FunctionalResult ← "PASS"

ELSE

 FunctionalResult ← "FAIL"

END IF

ELSE IF TestCase.Type = "BORROW" THEN

 result ← BorrowBook(


```
        TestCase.User,  
        TestCase.Book  
    )
```

```
IF result = SUCCESS THEN  
    FunctionalResult ← "PASS"  
ELSE  
    FunctionalResult ← "FAIL"  
END IF
```

```
ELSE IF TestCase.Type = "RETURN" THEN
```

```
    result ← ReturnBook(  
        TestCase.User,  
        TestCase.Book  
    )
```

```
IF result = SUCCESS THEN  
    FunctionalResult ← "PASS"  
ELSE  
    FunctionalResult ← "FAIL"  
END IF
```

```
END IF
```

ResponseTime $\leftarrow$ StopTimer()

NavigationSteps $\leftarrow$ CountNavigationSteps(TestCase.User)

AccessibilityResult $\leftarrow$ CheckAccessibility(
 TestCase.Interface
)

Store(ResponseTimes, ResponseTime)

Store(NavigationScores, NavigationSteps)

Store(AccessibilityResults,
 AccessibilityResult)

Log(
 TestCase.ID,
 FunctionalResult,
 ResponseTime,
 NavigationSteps,
 AccessibilityResult
)

END FOR

AverageResponseTime $\leftarrow$ Average(ResponseTimes)

AverageNavigation $\leftarrow$ Average(NavigationScores)

AccessibilityRate $\leftarrow$ CalculateAccessibilityRate(
AccessibilityResults
)

Display("Average Response Time:",
AverageResponseTime)

Display("Average Navigation Steps:",
AverageNavigation)

Display("Accessibility Rate:",
AccessibilityRate)

IF AverageResponseTime > ResponseThreshold THEN
Report("Performance improvement required")
END IF

IF AverageNavigation > NavigationThreshold THEN
Report("Navigation should be simplified")
END IF

IF AccessibilityRate < AccessibilityThreshold THEN
Report("Accessibility improvements required")

END IF

GenerateFinalQAUsabilityReport()

END Library_QA_Usability_Test

RUBRICS FOR CALCULATION:**Error Analysis Task**

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand